

Claude Code Hooks 완전 정리

1. 훅(Hook)이란?

훅은 Claude Code의 생명주기(lifecycle) 중 특정 시점에 자동으로 실행되는 셸 명령, HTTP 엔드포인트, MCP tool, 또는 LLM 프롬프트입니다.

핵심 철학: 시스템 프롬프트(CLAUDE.md)는 "요청"이고, ^{LLM프롬프트: LLM한테 넘어가는 text} 혹은 "보장"입니다.

- CLAUDE.md에 "커밋 전에 항상 lint를 실행해" → Claude가 잊거나 건너뛴 수 있는 ^{LLM} 제안
- `PreToolUse` 훅으로 `git commit`을 가로채 lint 체크 → Claude의 의지와 무관하게 ^{LLM} 강제되는 규칙

혹은 AI 외부에서 ^{무조건 한다} 결정론적으로 동작하므로, "반드시 지켜야 하는 규칙"은 CLAUDE.md가 아니라 훅으로 구현하는 것이 원칙입니다.

2. 생명주기(Lifecycle) 개요

이벤트는 발생 빈도에 따라 세 가지로 분류됩니다.

빈도	이벤트
세션당 1회	<code>SessionStart</code> , <code>SessionEnd</code>
턴(turn)당 1회 턴: 질문-답변 돌아옴.	<code>UserPromptSubmit</code> , <code>Stop</code> , <code>StopFailure</code>
tool 호출마다 (에이전틱 루프 내)	<code>PreToolUse</code> , <code>PostToolUse</code>

대략적인 흐름:

```
[Setup] → SessionStart
  → (턴 반복) UserPromptSubmit → UserPromptExpansion(슬래시 커맨드 시)
    → [에이전틱 루프: PreToolUse → PermissionRequest → PostToolUse
      → PostToolUseFailure → PostToolBatch → SubagentStart/Stop →
TaskCreated/Completed]
    → Stop 또는 StopFailure
  → TeammateIdle, PreCompact, PostCompact
→ SessionEnd
```

에이전틱 루프
외워두기.

이 외에 `WorktreeCreate/Remove`, `Notification`, `ConfigChange`, `InstructionsLoaded`, `CwdChanged`, `FileChanged`, `Elicitation/ElicitationResult`, `MessageDisplay` 등이 독립적으로 발생합니다.

3. 전체 이벤트 목록 (주요 이벤트)

이벤트	발생 시점
SessionStart	세션 시작/재개 시
Setup	--init-only, -p --init, -p --maintenance 실행 시 (1회성 준비 작업)
UserPromptSubmit	사용자가 프롬프트 제출, Claude 처리 전
UserPromptExpansion	슬래시 커맨드가 프롬프트로 확장되기 직전
PreToolUse	tool 호출 실행 전 (차단 가능)
PermissionRequest	권한 다이얼로그가 뜰 때
PermissionDenied	auto 모드 분류기가 tool 호출을 거부했을 때
PostToolUse	tool 호출 성공 후
PostToolUseFailure	tool 호출 실패 후
PostToolBatch	병렬 tool 호출 배치 완료 후, 다음 모델 호출 전
Notification	Claude Code가 알림을 보낼 때
MessageDisplay	어시스턴트 메시지 텍스트가 화면에 표시되는 동안
SubagentStart / SubagentStop	서브에이전트 생성/완료 시
TaskCreated / TaskCompleted	태스크 생성/완료 표시 시
Stop / StopFailure	Claude 응답 종료 시 / API 오류로 턴 종료 시
TeammateIdle	에이전트 팀 구성원이 유휴 상태로 전환되기 직전
InstructionsLoaded	CLAUDE.md 또는 규칙 파일 로드 시
ConfigChange	설정 파일 변경 시
CwdChanged	작업 디렉터리 변경 시 (예: cd 실행)
FileChanged	감시 중인 파일이 디스크에서 변경될 때
WorktreeCreate / WorktreeRemove	워크트리 생성/제거 시
PreCompact / PostCompact	컨텍스트 압축 전/후
Elicitation / ElicitationResult	MCP 서버가 사용자 입력을 요청할 때 / 응답 후

이벤트	발생 시점
SessionEnd	세션 종료 시

4. 설정(Configuration) 구조

혹 설정은 3단계 중첩 구조입니다.

1. 이벤트(event) 선택 - 예: PreToolUse

2. 매처 그룹(matcher group)으로 필터링 - 예: "Bash tool만" 세밀한 때

3. ~~혹~~ 핸들러(handler) 정의 - 실제 실행될 명령/MCP URL/prompt
처리기 다시 LLM 다녀옴.
한 번 더 태울 수 있음.

json settings.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"$FILE\""
          }
        ]
      }
    ]
  }
}
```

← 사용 후에

← 특정 파일

← 수정 or 삭제

← 포매팅

← 실행

← 수정되거나 쓰여진 파일

4.1 정의 위치와 범위

위치	범위	공유 가능 여부
~/.claude/settings.json	모든 프로젝트 적용	불가 (로컬 전용)
.claude/settings.json	단일 프로젝트 적용	가능 (커밋 가능)
.claude/settings.local.json	단일 프로젝트(개인용)	불가 (gitignore됨)
관리형(managed) 정책 설정	조직 전체	가능 (관리자 제어)
플러그인 hooks/hooks.json	플러그인 활성화 시	가능 (플러그인 번들)
스킬/서브에이전트 frontmatter	해당 컴포넌트 활성화 중	가능

관리자는 allowManagedHooksOnly로 사용자/프로젝트/플러그인 혹을 차단할 수 있습니다.

4.2 매처(Matcher) 패턴 세밀한 필터링

매처 값

평가 방식

"*", "", 생략 matcher 존재x

전체 매칭

영문자·숫자·[]·-·공백·,·|만 포함

정확한 문자열(들) 매칭. (Edit|Write), (Edit, Write) 모두가

그 외 문자 포함

JavaScript 정규식(비앵커링)으로 평가

- (Bash)는 Bash tool만 정확히 매칭
- (mcp__memory__.*처럼 MCP 서버의 모든 tool을 매칭하려면 (.*) 필수 ((mcp__memory)만 쓰면 정확 문자열로 취급되어 매칭 안 됨)
- (FileChanged), (StopFailure)는 좁은 문자 집합(영문자·숫자·[]·|)만 정확 매칭으로 처리

더 세밀한 필터링은 개별 핸들러의 (if) 필드(권한 규칙 문법, 예: ("Bash(git *)"), ("Edit(*.ts)"))로 가능합니다. tool 이벤트에서만 평가됩니다.

Bash(git:*)가
맞다고 함..

5. 후 핸들러 5가지 타입

타입

설명

command

셸 명령 실행. stdin으로 JSON 받고 exit code/stdout으로 응답

http

URL로 JSON POST. 응답 body가 stdout JSON과 동일 스키마
원격지

mcp_tool

이미 연결된 MCP 서버의 tool 호출

prompt

LLM에게 단일 턴 평가 요청 (yes/no 결정)

agent

Read/Grep/Glob 등을 쓸 수 있는 서브에이전트 생성 (실험적)

모든 매칭된 혹은 병렬로 실행되며, 동일한 핸들러는 자동으로 중복 제거됩니다(command는 명령 문자열+args 기준, http는 URL 기준).

5.1 Command 후 (type: "command")

type="command" command : ""

↑ Command 후

셸 명령을 실행합니다. 이벤트의 JSON 입력을 stdin으로 받고, exit code와 stdout을 통해 결과를 전달합니다.

전용 필드:

필드	필수	설명
<code>command</code>	예 (필수)	실행할 셸 명령. <code>args</code> 가 있으면 직접 실행할 실행 파일 경로
<code>args</code>	아니오 (옵션)	인자 목록. 있으면 <code>command</code> 가 실행 파일로 해석되어 셸 없이 직접 spawn됨
<code>async</code>	아니오 (옵션)	<code>true</code> 면 백그라운드에서 논블로킹으로 실행
<code>asyncRewake</code>	아니오 (옵션)	백그라운드 실행 + <code>exit 2</code> 시 Claude를 깨움 (<code>async</code> 자동 포함)
<code>shell</code>	아니오 (옵션)	"bash" 또는 "powershell" (기본값은 OS에 따라 자동 결정)

Exec form vs Shell form (중요한 구분):

- Exec form** (`args` 지정 시): 셸 없이 `command`를 실행 파일로 직접 spawn. 특수문자(`$`, 백틱 등)가 그대로 전달됨, 따옴표 처리 불필요
- Shell form** (`args` 생략 시): `command` 문자열이 셸(`sh -c`, Git Bash, PowerShell)에 전달되어 파이프, `&&` 리다이렉트, 그룹 등 셸 기능 사용 가능

```

json
{
  "type": "command",
  "command": "node",
  "args": ["${CLAUDE_PLUGIN_ROOT}/scripts/format.js", "--fix"]
}

```

5.2 HTTP 훅 (`type: "http"`)

지정한 URL로 이벤트 JSON을 **POST 요청 body**로 전송합니다. 응답 body가 `command` 훅의 `stdout` JSON과 동일한 스키마로 처리됩니다.

전용 필드:

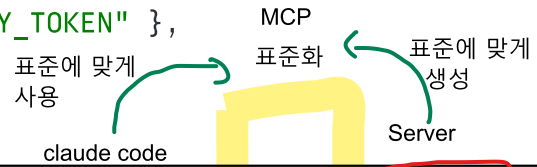
필드	필수	설명
<code>url</code>	예	POST 요청을 보낼 URL (서버의 위치)
<code>headers</code>	아니오	추가 HTTP 헤더 (<code>\$VAR_NAME</code> 형태로 환경변수 삽입 가능)
<code>allowedEnvVars</code>	아니오	헤더에 삽입 허용할 환경변수 이름 목록 (필수: 지정 안 하면 삽입 안 됨)

Command 훅과의 결정적 차이: 비-2xx 응답, 연결 실패, 타임아웃은 모두 **비차단(non-blocking) 오류**로 처리되어 실행이 계속됩니다. `exit code 2` 같은 강제 차단 메커니즘이 없으므로, tool 호출을 차단하려면 반드시 **2xx 응답 + `decision: "block"`** (또는 `permissionDecision: "deny"`) JSON body를 반환해야 합니다.

```

json
{
  "type": "http",
  "url": "http://localhost:8080/hooks/pre-tool-use",
  "timeout": 30,
  "headers": { "Authorization": "Bearer $MY_TOKEN" },
  "allowedEnvVars": ["MY_TOKEN"]
}

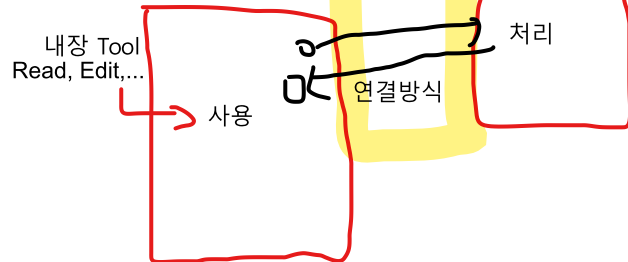
```



5.3 MCP tool 혹은 (type: "mcp_tool")

이미 연결된 MCP 서버의 tool을 호출합니다.
표준화된 규칙

전용 필드:



필드	필수	설명
server	예	설정된 MCP 서버 이름 (플러그인 서버는 plugin:<플러그인명>:<서버명> 형태)
tool	예	호출할 tool 이름
input	아니오	tool에 전달할 인자. "\${tool_input.file_path}" 처럼 혹은 입력 JSON에서 값 치환 가능

tool의 텍스트 출력은 command 혹은 stdout처럼 취급됩니다 — 유효한 JSON 출력이면 결정 (decision)으로 파싱되고, 아니면 그냥 텍스트로 표시됩니다.

제약: 서버가 연결되어 있지 않거나 tool이 (isError: true)를 반환하면 비차단 오류로 처리됩니다. (SessionStart) (Setup)은 보통 MCP 서버 연결 전에 발생하므로 첫 실행 시 "연결 안 됨" 오류가 날 수 있습니다.

```

json
{
  "type": "mcp_tool",
  "server": "my_server",
  "tool": "security_scan",
  "input": { "file_path": "${tool_input.file_path}" }
}

```

5.4 Prompt 혹은 (type: "prompt")

Claude 모델에게 단일 턴 평가를 요청합니다. 모델이 yes/no 결정을 JSON으로 반환합니다.

LLM 요청
전용 필드:

필드	필수	설명
<code>prompt</code>	예	모델에게 보낼 프롬프트 텍스트. <code>{\$ARGUMENTS}</code> 로 혹은 입력 JSON을 삽입 가능
<code>model</code>	아니오	평가에 사용할 모델 (기본값: 빠른 모델)

기본 타임아웃은 30초입니다.

단일처리에 좋음.

기존 필요할 경우 좋음

5.5 Agent 훅 (`type: "agent"`)

독립 컨텍스트로 움직임. (cf > Prompt 혹은 기존 컨텍스트 사용)

Read, Grep, Glob 같은 도구를 사용할 수 있는 서버 에이전트를 생성해 조건을 검증한 뒤 결정을 반환합니다. 실험적 기능이며 향후 변경될 수 있습니다.

Prompt 훅과 동일한 필드(`prompt`, `model`)를 사용하며, 기본 타임아웃은 60초입니다.

5.6 공통 필드 (모든 타입 공통)

필드	필수	설명
<code>type</code>	예	<code>"command"</code> <code>"http"</code> <code>"mcp_tool"</code> <code>"prompt"</code> <code>"agent"</code>
<code>if</code>	아니오	권한 규칙 문법 (<code>"Bash(git *)"</code>) 등으로 실행 조건 필터. tool 이벤트에서만 평가됨
<code>timeout</code>	아니오	기본값: <code>command/http/mcp_tool=600초</code> , <code>prompt=30초</code> , <code>agent=60초</code>
<code>statusMessage</code>	아니오	실행 중 표시할 스피너 메시지
<code>once</code>	아니오	<code>true</code> 면 세션당 한 번만 실행 후 제거 (스킬 frontmatter 훅에서만 적용)

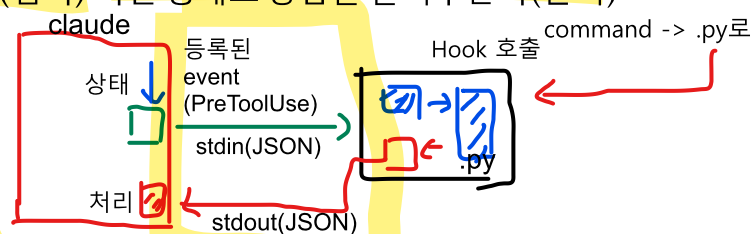
선택 기준 요약: 로컬 스크립트/셸 로직 → **command**, 외부 서비스 연동 → **http**, 이미 연결된 MCP 서버 활용 → **mcp_tool**, 자연어 기반 빠른 판단 → **prompt**, 파일 탐색이 필요한 복잡한 검증 → **agent**.

6. 입출력 계약 (I/O Contract) 상세

훅이 Claude Code로부터 어떤 형태로 데이터를 받고(입력) 어떤 형태로 응답을 돌려주는지(출력)에 대한 규약입니다.

6.1 입력(Input) 방식

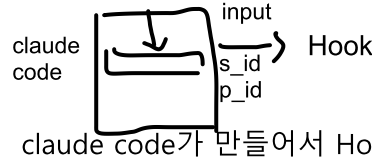
훅 핸들러 종류에 따라 입력 전달 방식이 다릅니다.



py로 해야 크로스 플랫폼 가능

- **Command** **혹:** JSON 데이터가 **stdin**으로 전달됩니다.
- **HTTP** **혹:** 같은 JSON이 **POST 요청 body**로 전달됩니다 (`Content-Type: application/json`).
- **MCP tool** **혹:** `{tool_input.file_path}` 같은 경로 치환으로 JSON 필드 일부를 tool 인자에 주입할 수 있습니다.
- **Prompt/Agent** **혹:** `{arguments}` 플레이스홀더로 입력 JSON을 프롬프트에 삽입합니다.

공통 입력 필드 (모든 이벤트 공통):



claude code가 만들어서 Hook한테 넘겨줌.

필드	설명	
<code>session_id</code>	현재 세션 식별자	예. agent teams에서 필요. Multi Agent: Session 1개, Context 여러 개 Agent Teams: Session 여러 개, LLM 여러 개
<code>prompt_id</code>	현재 처리 중인 사용자 프롬프트의 UUID	context
<code>transcript_path</code>	대화 JSON 파일 경로 (비동기 기록이라 최신 턴은 누락될 수 있음)	Context 주입 : Context에 직접 넣어줄 수도 있음.
<code>cwd</code>	혹 실행 시점의 작업 디렉터리	
<code>permission_mode</code>	<code>default</code> / <code>plan</code> / <code>acceptEdits</code> / <code>auto</code> / <code>dontAsk</code> / <code>bypassPermissions</code>	plan 모드에서는 코드 실행이 안 됨.
<code>effort</code>	현재 턴의 effort 레벨	
<code>hook_event_name</code>	발생한 이벤트 이름	

여기에 **이벤트별 필드**가 추가됩니다. 예를 들어 `PreToolUse`는 `tool_name`, `tool_input`을 포함합니다:

```
json
{
  "session_id": "abc123",
  "hook_event_name": "PreToolUse",
  "tool_name": "Bash",
  "tool_input": { "command": "npm test" }
}
```

6.2 출력(Output) 방식 — 두 채널

파이썬 파일

혹은 **exit code** (굵은 채널)와 **stdout의 JSON** (정교한 채널) 두 가지로 응답합니다. 이 둘은 함께 쓰지 않고 하나만 선택합니다.

Exit code 방식

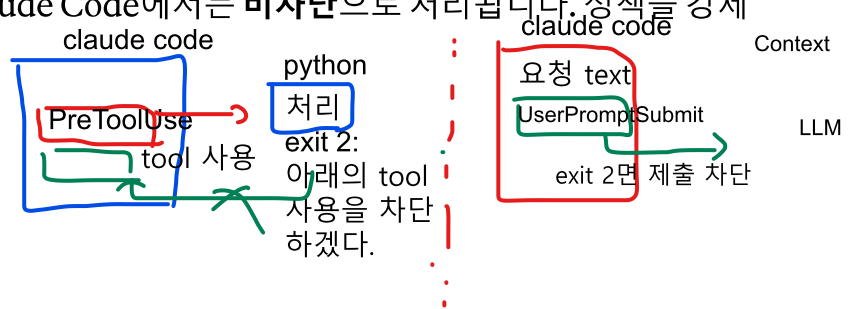
- **exit 0:** 성공. stdout의 JSON을 파싱함 (대부분 이벤트는 디버그 로그로만 가고, `UserPromptSubmit` / `UserPromptExpansion` / `SessionStart`만 Claude 컨텍스트에 반영됨) 주입

- **exit 2:** 차단 오류. `stdout`의 JSON은 무시되고, `stderr`가 Claude에게 오류 메시지로 전달됨. 효과는 이벤트마다 다름 (`PreToolUse`는 tool 호출 차단, `UserPromptSubmit`은 프롬프트 거부 등)
- **그 외 exit code:** 대부분 이벤트에서 비차단 오류. `stderr` 첫 줄이 트랜스크립트에 표시되고 실행은 계속됨

⚠ 주의: exit 1은 관습적 실패 코드지만 Claude Code에서는 **비차단**으로 처리됩니다. 정책을 강제하려면 반드시 **exit 2**를 써야 합니다.

JSON 출력 방식 (exit 0 + JSON)

범용 필드 (모든 이벤트 공통):



필드	기본값	설명
<code>continue</code>	<code>true</code>	<code>false</code> 면 Claude가 전체 처리를 즉시 중단 (다른 결정 필드보다 우선)
<code>stopReason</code>	없음	<code>continue: false</code> 일 때 사용자에게 보여줄 메시지
<code>suppressOutput</code>	<code>false</code>	<code>stdout</code> 을 트랜스크립트에서 숨김
<code>systemMessage</code>	없음	사용자에게 보여줄 경고 메시지
<code>terminalSequence</code>	없음	터미널 알림/제목 변경용 이스케이프 시퀀스

이벤트별 결정 패턴 — `hookSpecificOutput`에 `hookEventName`을 명시해야 합니다:

- **Top-level `decision`** (`UserPromptSubmit`, `PostToolUse`, `Stop` 등): `{"decision": "block", "reason": "..."}`
- **`PreToolUse`**: `hookSpecificOutput.permissionDecision` (`allow`/`deny`/`ask`/`defer`) + `permissionDecisionReason`, 또는 `updatedInput`으로 tool 인자 자체를 재작성
- **`PostToolUse`**: `updatedToolOutput`으로 tool 결과 자체를 교체 가능
- **`PermissionRequest`**: `hookSpecificOutput.decision.behavior` (`allow`/`deny`)
- **`SessionStart`**: `additionalContext`, `initialUserMessage`, `sessionTitle`, `watchPaths`, `reloadSkills`

예시 (PreToolUse 차단):

json

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Database writes are not allowed"
  }
}
```

python에서는 딕셔너리 -> JSON 변환하여 반환

6.3 HTTP hook의 응답 처리 (Command hook과 다름)

- **2xx + 빈 body**: 성공 (exit 0과 동일)
- **2xx + plain text body**: 성공, 텍스트가 컨텍스트로 추가됨
- **2xx + JSON body**: 성공, JSON 출력 스키마로 파싱
- **비 2xx / 연결 실패 / 타임아웃**: 항상 **비차단** 오류로 처리 (exit 2 같은 강제 차단 불가) — 차단하려면 반드시 2xx + `{decision: "block"}` JSON을 반환해야 함

6.4 `additionalContext` — Claude에게 정보 전달

`hookSpecificOutput.additionalContext`는 시스템 리마인더 형태로 Claude 컨텍스트에 삽입됩니다(채팅 메시지는 안 보임). 이벤트에 따라 삽입 위치가 다릅니다:

- `SessionStart`/`Setup`/`SubagentStart`: **대화 시작 시점, 첫 프롬프트 이전**
- `UserPromptSubmit`/`UserPromptExpansion`: 제출된 프롬프트 옆 **기존 대화** 새 대화
- `PreToolUse`/`PostToolUse`/`PostToolUseFailure`/`PostToolBatch`: **tool 결과 옆**
- `Stop`/`SubagentStop`: **턴 종료 시점** (대화가 계속되어 Claude가 피드백에 반응 가능)

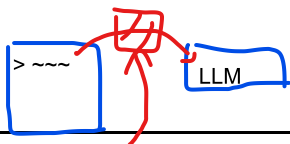
기본: claude code에서 일하다가 이벤트 만나면 흑으로 이동. input은 claude code가 만들어줌. hook에서 처리 후 output으로 돌려줌.

6.5 핵심 원칙 요약

1. 입력은 **stdin**(또는 HTTP body)의 **JSON**, 출력은 exit code + stdout JSON(또는 HTTP status + body)
2. **exit 2만**이 진짜 "차단" 신호 (이벤트가 차단 가능한 경우에 한함) `stderr`
3. **JSON**은 exit 0일 때만 파싱됨
4. stdout에는 **JSON만 있어야 함** (셀 프로파일이 부가 텍스트를 출력하면 파싱 실패)
5. 이벤트마다 지원하는 결정 필드가 다르므로, hook을 작성하기 전에 해당 이벤트의 "Decision control" 표를 반드시 확인해야 함

6.1 이벤트별 exit code 2의 효과

이벤트	차단 가능?	exit 2 효과
<code>PreToolUse</code>	✓	tool 호출 차단
<code>PermissionRequest</code>	✓	권한 거부

이벤트	차단 가능?	exit 2 효과
 UserPromptSubmit	 UserPromptExpansion	프롬프트 처리 차단 및 삭제
Stop / SubagentStop	✓	종료 방지, 대화 계속
TeammateIdle	✓	유휴 전환 방지
TaskCreated / TaskCompleted	✓	생성 롤백 / 완료 방지
ConfigChange	✓	설정 변경 차단 (policy_settings 제외)
PreCompact	✓	압축 차단
Elicitation / ElicitationResult	✓	요청 거부 / 응답 차단
WorktreeCreate	✓	워크트리 생성 실패 (0이 아닌 모든 exit code)
PostToolUse, PostToolUseFailure	✗	stderr만 Claude에게 표시 (이미 실행 됨)
PermissionDenied	✗	exit code 무시됨. retry: true로 재 시도 유도
SessionStart, Setup, Notification, SessionEnd 등	✗	stderr는 사용자에게만 표시

6.2 결정(Decision) 제어 패턴 정리

이벤트 그룹	패턴	핵심 필드
UserPromptSubmit, PostToolUse, Stop 등	top-level decision	decision: "block", reason
PreToolUse	hookSpecificOutput	permissionDecision(allow/de updatedInput)
PermissionRequest	hookSpecificOutput	decision.behavior(allow/den updatedInput)
PostToolUse	결과 재작성	updatedToolOutput

이벤트 그룹	패턴	핵심 필드
SessionStart/Setup/SubagentStart	컨텍스트만	additionalContext, initialUserMessage, sessionWatchPaths, reloadSkills
WorktreeCreate	경로 반환	command는 stdout에 경로 출력, hookSpecificOutput.worktree
MessageDisplay	화면 표시만	displayContent (트랜스크립트는 내용은 원본 유지)
WorktreeRemove, Notification, SessionEnd 등	없음	로깅·정리 등 부수효과 용도

7. 자주 쓰는 실전 패턴

7.1 파일 편집 후 자동 포매팅 (PostToolUse)

```

json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "npx prettier --write \"$(jq -r .tool"
        ]
      }
    ]
  }
}

```

7.2 위험한 Bash 명령 차단 (PreToolUse)

```

bash

```

```
#!/bin/bash
command=$(jq -r '.tool_input.command')
if echo "$command" | grep -q 'rm -rf'; then
  jq -n '{
    hookSpecificOutput: {
      hookEventName: "PreToolUse",
      permissionDecision: "deny",
      permissionDecisionReason: "Destructive command blocked by hook"
    }
  }'
else
  exit 0
fi
```

7.3 테스트 통과 전까지 종료 방지 ((Stop))

```
bash

#!/bin/bash
input=$(cat)
if [ "$(echo "$input" | jq -r '.stop_hook_active')" = "true" ]; then
  exit 0 # 이미 이 훅 때문에 계속 진행 중 → 무한 루프 방지
fi
if ! npm test --silent > /tmp/log 2>&1; then
  jq -n '{decision: "block", reason: "Tests are failing."}'
fi
```

⚠ v2.1.143부터, 같은 터에서 (Stop) 훅이 8회 연속 차단하면 세션이 강제 종료됩니다(무한 루프 방지 안전장치).

7.4 세션 시작 시 컨텍스트 주입 ((SessionStart))

```
bash

#!/bin/bash
echo "Current branch: $(git branch --show-current)"
echo "Uncommitted files: $(git status --porcelain | wc -l)"
```

(SessionStart는 stdout이 그대로 컨텍스트로 들어가므로 JSON 없이도 동작)

7.5 .env 파일 접근 차단

(PreToolUse)를 (Edit|Write|Read)에 매칭시켜 (tool_input.file_path)가 (.env)를 포함하면 exit 2로 차단. 권한 규칙(.claudeignore)만으로는 인덱싱·검색을 통한 우회가 가능하므로 훅이 유일한 확실한 차단 방법입니다.

8. `/hooks` 메뉴와 관리

- `/hooks` 입력 시 현재 등록된 모든 hooks를 읽기 전용으로 조회 가능 (이벤트별 개수, 매치, 출처: User/Project/Local/Plugin/Session/Built-in)
 - hooks 추가/수정/삭제는 설정 JSON을 직접 편집(또는 Claude에게 요청)해야 함 자동화 계획 미리 세워서 시켜야 함.
 - 모든 hooks를 일시적으로 끄려면 `"disableAllHooks": true`
 - 개별 hooks만 끄는 기능은 없음 — 설정에서 제거해야 함
-

9. 주의사항 총정리

1. ^{무시} `exit 1`은 차단 안 됨 — 반드시 `exit 2`를 사용
 2. `exit code`와 JSON을 섞어 쓰지 말 것 — `exit 2`면 `stdout JSON`은 무조건 무시됨
 3. `stdout`에는 JSON만 — 셸 프로파일이 부가 출력을 하면 파싱 실패
 4. macOS/Linux에서 hooks는 controlling terminal 없이 실행됨 → `/dev/tty` 직접 접근 불가, `terminalSequence` 필드 사용
 5. `if` 필드는 best-effort이므로, 강제적 허용/거부는 hook보다 권한 시스템을 사용할 것
 6. 출력 문자열(`additionalContext`, `systemMessage` 등)은 10,000자로 제한됨 (초과 시 파일로 저장되고 미리보기만 전달)
-

참고 문서: <https://code.claude.com/docs/en/hooks> (Hooks reference) ·

<https://code.claude.com/docs/en/hooks-guide> (퀵스타트 가이드)